JAVA PROGRAMMING LAB

EXPERIMENT-1

1a) Write a JAVA program to display default value of all primitive data type of JAVA

Aim: Write a JAVA program to display default value of all primitive data type of JAVA

PROGRAM:

```
class DefaultValues
{
   byte b;
   short s;
   int i;
   long l;
   float f;
   double d;
   char c;
   boolean bool;
          public static void main(String[] args)
 {
      DefaultValues obj = new DefaultValues();
      System.out.println("Default value of byte: " + obj.b);
      System.out.println("Default value of short: " + obj.s);
      System.out.println("Default value of int: " + obj.i);
      System.out.println("Default value of long: " + obj.l);
      System.out.println("Default value of float: " + obj.f);
      System.out.println("Default value of double: " + obj.d);
      System.out.println("Default value of char: " + obj.c);
      System.out.println("Default value of boolean: " + obj.bool);
   }
}
```

Output :

Default value of byte: 0

Default value of short: 0

Default value of int: 0

Default value of long: 0

Default value of float: 0.0

Default value of double: 0.0

Default value of char: (a blank space)

Default value of Boolean: false

Result: The program is executed successfully.

1 b) Write a java program that display the roots of a quadratic equation ax2+bx=0. Calculate the discriminate D and basing on the value of D, describe the nature of the root.

Aim: Write a java program that display the roots of a quadratic equation ax2+bx=0. Calculate the discriminate D and basing on the value of D, describe the nature of the root.

PROGRAM:

```
import java.util.Scanner;
public class QuadraticEquation
  {
     public static void main(String[] args)
       {
         Scanner sc = new Scanner(System.in);
         System.out.print("Enter the value of a: ");
       double a = sc.nextDouble();
       System.out.print("Enter the value of b: ");
      double b = sc.nextDouble();
     System.out.print("Enter the value of c: ");
    double c = sc.nextDouble();
   double D = b * b - 4 * a * c;
```

```
                if (D > 0)
                    {
                     double root1 = (-b + Math.sqrt(D)) / (2 * a);
                    double root2 = (-b - Math.sqrt(D)) / (2 * a);
                   System.out.println("Roots are real and distinct.");
                   System.out.println("Root 1: " + root1);
                  System.out.println("Root 2: " + root2);
                  }
           else if (D == 0)
         {
        double root = -b / (2 * a);
        System.out.println("Roots are real and equal.");
        System.out.println("Root: " + root);
     }
    else
        {
        double realPart = -b / (2 * a);
        double imaginaryPart = Math.sqrt(-D) / (2 * a);
        System.out.println("Roots are complex.");
        System.out.println("Root 1: " + realPart + " + " + imaginaryPart + "i");
        System.out.println("Root 2: " + realPart + " - " + imaginaryPart + "i");
     }
  }  }
```

OUTPUT:

Example input:

Enter the value of a: 1

Enter the value of b: -3

Enter the value of c: 2

Example output:
Roots are real and distinct.

Root 1: 2.0

Root 2: 1.0

Result: The program is executed successfully.

EXPERIMENT-2

2a) Write a JAVA program to search for an element in a given list of elements using binary search mechanism.

Aim: Write a JAVA program to search for an element in a given list of elements using binary search mechanism.

Program :

```
import java.util.Scanner;
public class BinarySearch
{
    public static void main(String[] args)
    {
Scanner sc = new Scanner(System.in);
System.out.print("Enter number of elements: ");
int n = sc.nextInt();
int[] arr = new int[n];
System.out.println("Enter " + n + " sorted elements:");
for (int i = 0; i < n; i++) {
    arr[i] = sc.nextInt();
}
   System.out.print("Enter element to search: ");
int key = sc.nextInt();
 int result = binarySearch(arr, key);
if (result == -1) {
    System.out.println("Element not found.");
} else {
```

```
            System.out.println("Element " + key + " found at index " + result + ".");
        }
        }
    // Binary Search function
    public static int binarySearch(int[] arr, int key) {
        int low = 0, high = arr.length - 1;
        while (low <= high) {
            int mid = (low + high) / 2;
            if (arr[mid] == key) {
                return mid;
            } else if (arr[mid] < key) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return -1;
    }
}
```

OUTPUT :

Example input:

Enter number of elements: 5

Enter 5 sorted elements: 10 20 30 40 50

Enter element to search: 30

Example output:

Element 30 found at index 2

Result: The program is executed successfully.

.

2b) Write a JAVA program to sort for an element in a given list of elements using bubble sort

Aim: Write a JAVA program to sort for an element in a given list of elements using bubble sort

Program:

```
import java.util.Scanner;
public class BubbleSort
    {
        public static void main(String[] args)
        {
     Scanner sc = new Scanner(System.in);
    System.out.print("Enter number of elements: ");
    int n = sc.nextInt();
    int[] arr = new int[n];
   System.out.println("Enter " + n + " elements:");
  for (int i = 0; i < n; i++)
     {
         arr[i] = sc.nextInt();
    }
     // Bubble sort algorithm
     for (int i = 0; i < n - 1; i++)
        {
        for (int j = 0; j < n - i - 1; j++)
       {
              if (arr[j] > arr[j + 1])
              {
           // Swap arr[j] and arr[j+1]
       int temp = arr[j];
       arr[j] = arr[j + 1];
       arr[j + 1] = temp;
```

```
            }
          }
        }
        // Print sorted array
        System.out.println("Sorted list:");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

OUTPUT:

Example input:

Enter number of elements: 5

Enter 5 unsorted elements: 50 20 40 10 30

Example output:

Sorted list: 10 20 30 40 50

Result: The program is executed successfully.

2 c) Write a JAVA program using StringBuffer to delete, remove character.

AIM: Write a JAVA program using StringBuffer to delete, remove character

Program:

```
import java.util.Scanner;
        public class StringBufferExample
         {
            public static void main(String[] args)
            {
              Scanner sc = new Scanner(System.in);
```

```
                System.out.print("Enter a string: ");
                String input = sc.nextLine();
                System.out.print("Enter index of character to remove: ");
                int index = sc.nextInt();
            // Use StringBuffer to remove character at given index
            StringBuffer sb = new StringBuffer(input);
            sb.deleteCharAt(index);
            // Display the modified string
            System.out.println("String after deletion: " + sb);
        }
}
```

OUTPUT:

Example input:

Enter a string: Hello World

Enter index of character to remove: 5

Example output:

String after deletion: HelloWorld

Result: The program is executed successfully.

EXPERIMENT-3

3a) Write a JAVA program to implement class mechanism. Create a class, methods and invoke them inside main method.

AIM : Write a JAVA program to implement class mechanism. Create a class, methods and invoke them inside main method.

Program:

```
class SimpleOperations
   {
    // Method to add two numbers
   public int add(int a, int b)
   {
       return a + b;
       }
    public int multiply(int a, int b)
    {
      return a * b;
       }
public static void main(String[] args)
{
       // Create an object of the class
    SimpleOperations obj = new SimpleOperations();
       // Invoke methods
   int sum = obj.add(10, 5);
    int product = obj.multiply(10, 5);
 // Display results
System.out.println("Sum: " + sum);
System.out.println("Product: " + product);
  }
}
```

OUTPUT :

```
Example output:
Sum: 15
Product: 50
```

Result: The program is executed successfully.

3 b) Write a JAVA program implement method overloading

AIM: Write a JAVA program implement method overloading

Program:

```
class OverloadingExample
        {
            public int add(int a, int b)
                {
                    return a + b;
                }
             Public int add(int a, int b, int c)
              {
                   return a + b + c;
            }
         public static void main(String[] args)
             {
             OverloadingExample obj = new OverloadingExample();
           int sum1 = obj.add(10, 20);
        int sum2 = obj.add(5, 15, 10);
       System.out.println("Sum of 2 integers: " + sum1);
      System.out.println("Sum of 3 integers: " + sum2);
}       }
```

OUTPUT :

Example input:

Adding 2 integers: 10, 20

Adding 3 integers: 5, 15, 10

Example output:

Sum of 2 integers: 30

Sum of 3 integers: 30

Result: The program is executed successfully.

3c) Write a JAVA program to implement constructor

AIM: Write a JAVA program to implement constructor

Program:

```
class Person
   {
      String name;
       int age;
// Constructor to initialize name and age
     public Person(String name, int age)
          {
          this.name = name;
          this.age = age;
          }
      public void display() {
     System.out.println("Name: " + name);
    System.out.println("Age: " + age);
         }
   public static void main(String[] args)
        {
          // Create an object of the class
          Person person = new Person("deepika", 25);

            // Display details
      person.display();
   }
}
```

OUTPUT:

Example input:

Name: deepika , Age: 25

Example output:

Name: deepika, Age: 25

Result: The program is executed successfully.

3 d) Write a JAVA program to implement constructor overloading.

AIM: Write a JAVA program to implement constructor overloading.

Program:

```
class Person
 {
  String name;
    int age;
  // Default constructor
  public Person() {
this.name = "Default";
this.age = 0;
}
public Person(String name, int age)
     {
     this.name = name;
      this.age = age;
     }
  Public void display()
     {
       System.out.println("Name: " + name);
System.out.println("Age: " + age);
```

```
    }
    public static void main(String[] args)
        {
      // Using default constructor
      Person person1 = new Person();
      person1.display();

      // Using parameterized constructor
      Person person2 = new Person("deepika", 25);
      person2.display();
    }
}
```

OUTPUT:

Example input:

Using default constructor

Using parameterized constructor: Name: deepika, Age: 25

Example output:

Name: Default, Age: 0

Name: deepika, Age: 25

Result: The program is executed successfully.

EXPERIMENT-4

4a) Write a JAVA program to implement Single Inheritance

AIM: Write a JAVA program to implement Single Inheritance

Program:

```
class Person
    {
```

```
        String name;
          int age;


                  // Method to set person details
      public void setDetails(String name, int age)
           {
                  this.name = name;
                   this.age = age;
           }
      // Method to display person details
      public void display()
        {
          System.out.println("Name: " + name + ", Age: " + age);
         }
    }
// Derived class
   class Student extends Person
   {
       // Additional methods or properties can be added here if needed
   }
          public class SingleInheritance {
           public static void main(String[] args) {
          // Create an object of the derived class
                  Student student = new Student();
 // Set and display person details
 student.setDetails("deepika", 25);
 student.display();
  }
}
```

OUTPUT:

Example input:

Enter name: deepika

Enter age: 25

Example output:

Name: deepika, Age: 25

Result: The program is executed successfully.

4b) Write a JAVA program to implement multi level Inheritance

AIM: Write a JAVA program to implement multi level Inheritance

Program:

```
class Person
    {
                String name;
                 int age;
         // Method to set person details
          public void setPersonDetails(String name, int age)
           {
              this.name = name;
            this.age = age;
           }
             public void displayPersonDetails() {
      System.out.println("Name: " + name + ", Age: " + age);
   }
}
          // Derived class
              class Employee extends Person
                  {
               String job;
```

```
        public void setJobDetails(String job)
         {
         this.job = job;
         }
        public void displayJobDetails()
           {
                displayPersonDetails();
            System.out.println("Job: " + job);
      }
}
   // Further derived class
class Manager extends Employee
        {
  String department;
     public void setDepartment(String department) {
     this.department = department;
  }
      public void displayAllDetails()
          {
            displayJobDetails();
          System.out.println ("Department: " + department);
  }
}
     public class MultiLevelInheritance
           {
             public static void main(String[] args)
               {
     // Create an object of the Manager class
     Manager manager = new Manager();
     // Set and display all details
     manager.setPersonDetails("deepika", 25);
```

```
        manager.setJobDetails("Engineer");
        manager.setDepartment("R&D");
        manager.displayAllDetails();
    }
}
```

OUTPUT:

Example input:

Enter name: deepika

Enter age: 25

Enter job: Engineer

Example output:

Name: deepika, Age: 25, Job: Engineer

Result: The program is executed successfully.

4c) Write a JAVA program for abstract class to find areas of different shapes

AIM: Write a JAVA program for abstract class to find areas of different shapes

Program:

```
import java.util.Scanner;
// Abstract class Shape
abstract class Shape
{
// Abstract method to calculate area
abstract double area();
}
// Circle class extending Shape
class Circle extends Shape
```

```
            {
             double radius;
            // Constructor to initialize radius
          public Circle(double radius)
              {
                this.radius = radius;
          }
        // Implement area method for circle
    @Override
    double area() {
       return  Math.PI * radius * radius;
    }     }
// Rectangle class extending Shape
class Rectangle extends Shape
           {
    double length, width;
          public Rectangle(double length, double width)
                {
                    this.length = length;
                   this.width = width;
                    }
          // Implement area method for rectangle
    @Override
    double area()
           {
       return length * width;
    }
}
  public class AbstractClassExample
                 {
    public static void main(String[] args)
```

```
                {
        Scanner sc = new Scanner(System.in);
            // Input for circle
        System.out.print("Enter radius of circle: ");
        double radius = sc.nextDouble();
        Circle circle = new Circle(radius);
             // Input for rectangle
        System.out.print("Enter length and width of rectangle: ");
        double length = sc.nextDouble();
        double width = sc.nextDouble();
        Rectangle rectangle = new Rectangle(length, width);
        System.out.println("Area of Circle: " + circle.area());
        System.out.println("Area of Rectangle: " + rectangle.area());
    }
}
```

OUTPUT :

Example input:

Enter radius of circle: 5

Enter length and width of rectangle: 4 6

Example output:

Area of Circle: 78.54

Area of Rectangle: 24

Result: The program is executed successfully.

EXPERIMENT-5

5a) Write a JAVA program give example for “super” keyword.

Aim: Write a JAVA program give example for “super” keyword.

Program:

```
class Person {
    String name;
```

```
    int age;
    // Constructor of base class
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

// Derived class
class Employee extends Person {
    String job;

    // Constructor of derived class
    public Employee(String name, int age, String job) {
        super(name, age); // Call to base class constructor using super
        this.job = job;
    }

    public void displayDetails() {
        super.display(); // Call to base class method using super
        System.out.println("Job: " + job);
    }
}

public class SuperKeywordExample {
    public static void main(String[] args) {
```

```
        // Create an object of the derived class
        Employee emp = new Employee("John", 45, "Engineer");

        // Display details
        emp.displayDetails();
    }
}
```

Output:

Example input:

Enter name: John

Enter age: 45

Enter job: Engineer

Example output:

Name: John, Age: 45, Job: Engineer

Result: The program is executed successfully.

5b) Write a JAVA program to implement Interface. What kind of Inheritance can be achieved?

Aim: Write a JAVA program to implement Interface. What kind of Inheritance can be achieved?

Program:

```
interface Shape {
    double area();
}
// Implementing Circle class
class Circle implements Shape {
    double radius;

    public Circle(double radius) {
        this.radius = radius;
```

```
    }

    @Override
    public double area() {
        return Math.PI * radius * radius;
    }
}

// Implementing Rectangle class
class Rectangle implements Shape {
    double length, width;

    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    @Override
    public double area() {
        return length * width;
    }
}

public class InterfaceExample {
    public static void main(String[] args) {
        // Create objects of implementing classes
        Shape circle = new Circle(5);
        Shape rectangle = new Rectangle(4, 6);

        // Display areas
```

```
        System.out.println("Area of Circle: " + circle.area());
        System.out.println("Area of Rectangle: " + rectangle.area());
    }
}
```

Output:

Input: Implementing class behavior as defined in the interface.

Example output:

Area of Circle: 78.54

Area of Rectangle: 24

Result: The program is executed successfully.

5c) Write a JAVA program that implements Runtime polymorphism

Aim: Write a JAVA program that implements Runtime polymorphism

Program:

```
class Animal {
    // Method to be overridden
    public void sound() {
        System.out.println("Animal makes a sound");
    }
}
// Derived class Dog
class Dog extends Animal {
    @Override
    public void sound() {
        System.out.println("Dog makes a sound: Bark");
    }
}
```

```
// Derived class Cat
class Cat extends Animal {
    @Override
    public void sound() {
        System.out.println("Cat makes a sound: Meow");
    }
}
public class RuntimePolymorphismExample {
    public static void main(String[] args) {
        // Create objects of child classes
        Animal dog = new Dog();
        Animal cat = new Cat();
        // Runtime polymorphism: method call decided at runtime
        dog.sound();  // Dog's sound method is called
        cat.sound();  // Cat's sound method is called
    }  }
```

Output:

Input**:** Different objects of child classes invoking the same parent class method.

Example output:

Dog makes sound: Bark

Cat makes sound: Meow

Result: The program is executed successfully.

EXPERIMENT-6

A)Write a JAVA program that describes exception handling mechanism

Aim: To write a JAVA program that describes exception handling mechanism

Program:

```
public class Main{
   static void divide(int a, int b) throws ArithmeticException {
      if (b == 0) {
         throw new ArithmeticException("Division by zero is not allowed");
      }
      System.out.println("Result: " + (a / b));
   }
   public static void main(String[] args) {
      try {
         int arr[] = {10, 20, 30};
         System.out.println("Array element: " + arr[2]);
         divide(10, 0);
      } catch (ArithmeticException e) {
         System.out.println("Caught ArithmeticException: " + e.getMessage());
      } catch (ArrayIndexOutOfBoundsException e) {
         System.out.println("Caught ArrayIndexOutOfBoundsException: " +
e.getMessage());
      } catch (Exception e) {
         System.out.println("Caught Exception: " + e.getMessage());
      } finally{
         System.out.println("Finally block executed (cleanup code).");
      }
      System.out.println("Program continues after exception handling...");
   }
}
```

Output:

Array element: 30
Caught ArithmeticException: Division by zero is not allowed
Finally block executed (cleanup code).
Program continues after exception handling…

Result:

Thus the java program that describes exception handling mechanism is successfully executed.

B) Write a JAVA program Illustrating Multiple catch clauses

Aim: To write a JAVA program Illustrating Multiple catch clauses

Program:

```
public class Main {
    public static void main(String[] args) {
        try {
            int[] numbers = {1, 2, 3};
            System.out.println(numbers[5]);
            int result = 10 / 0;
        }
        catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Array error: " + e.getMessage());
        }
        catch (ArithmeticException e) {
         System.out.println("Math error: " + e.getMessage());
        }
        catch (Exception e) {
      System.out.println("Some other exception: " + e.getMessage());
        }
        finally {
            System.out.println("This block always executes.");
        }
```

```
            System.out.println("Program continues...");
        }
        }
```

Output:

```
Array error: Index 5 out of bounds for length 3
This block always executes.
Program continues...
```

Result:

Thus the JAVA program Illustrating Multiple catch clauses is successfully executed.

C) Write a JAVA program for creation of Java Built-in Exceptions

Aim: To write a JAVA program for creation of Java Built-in Exceptions

Program:

```
public class Main {
    public static void main(String[] args) {
        try {
            int a = 10, b = 0;
            int result = a / b;
            System.out.println("Result: " + result);
            int arr[] = {1, 2, 3};
            System.out.println("Accessing 5th element: " + arr[4]);
            String str = null;
            System.out.println("Length of string: " + str.length());
        } catch (ArithmeticException e) {
            System.out.println("Caught ArithmeticException: " + e.getMessage());
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Caught ArrayIndexOutOfBoundsException: " + e.getMessage());
        } catch (NullPointerException e) {
            System.out.println("Caught NullPointerException: " + e.getMessage());
        } catch (Exception e) {
```

```
            System.out.println("Caught Exception: " + e.getMessage());
        } finally {
            System.out.println("Finally block executed (cleanup code).");
        }
    }
}
```

Output:

```
Caught ArithmeticException: / by zero
Finally block executed (cleanup code).
```

Result:

Thus the JAVA program for creation of Java Built-in Exceptions is successfully executed.

D) Write a JAVA program for creation of User Defined Exception

Aim: To write a JAVA program for creation of User Defined Exception

Program:

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}
public class Main {
    static void checkAge(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Age must be 18 or above to vote!");
        } else {
            System.out.println("You are eligible to vote.");
        }
    }
    public static void main(String[] args) {
```

```
        try {
            checkAge(15);
        } catch (InvalidAgeException e) {
            System.out.println("Caught Exception: " + e.getMessage());
        } finally {
            System.out.println("Program finished.");
        }
    }
}
```

Output:

Caught Exception: Age must be 18 or above to vote!
Program finished.

Result:
Thus the JAVA program for creation of User Defined Exception is successfully executed.

EXPERIMENT- 7

A) Write a JAVA program that creates threads by extending Thread class.First thread display"Good Morning "every 1 sec, the second thread displays "Hello "every 2 seconds and the thirddisplay "Welcome" every 3 seconds,(Repeat the same by implementing Runnable)

Aim: To write a JAVA program that creates threads by extending Thread class

Program:

```
class GoodMorningThread extends Thread {
    public void run() {
        try {
            for (int i = 1; i <= 3; i++) {
                System.out.println("Good Morning");
                Thread.sleep(1000); // 1 second
            }
        } catch (InterruptedException e) {
```

```
            System.out.println(e);
        }
    }
}
class HelloThread extends Thread {
    public void run() {
        try {
            for (int i = 1; i <= 3; i++) {
                System.out.println("Hello");
                Thread.sleep(2000); // 2 seconds
            }
        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
}
class WelcomeThread extends Thread {
    public void run() {
        try {
            for (int i = 1; i <= 3; i++) {
                System.out.println("Welcome");
                Thread.sleep(3000); // 3 seconds
            }
        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
}
public class Main {
    public static void main(String[] args) {
```

```
        new GoodMorningThread().start();
        new HelloThread().start();
        new WelcomeThread().start();
    }
}
```

Output:

```
Good Morning
Hello
Welcome
Good Morning
Good Morning
Hello
Welcome
Hello
Welcome
```

Result:

Thus the JAVA program that creates threads by extending Thread class is successfully executed.

B) Write a program illustrating is Alive and join ()

Aim: To write a program illustrating is Alive and join ().

Program:

```
class MyThread extends Thread {
    public void run() {
        try {
            for (int i = 1; i <= 3; i++) {
                System.out.println(getName() + " - Count: " + i);
                Thread.sleep(500); // pause for 0.5 sec
            }
        } catch (InterruptedException e) {
```

```
            System.out.println(e);
        }
    }
}
public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();
        System.out.println("Before starting:");
        System.out.println("t1 is alive? " + t1.isAlive()); // false
        System.out.println("t2 is alive? " + t2.isAlive()); // false
        // start threads
        t1.start();
        t2.start();
        System.out.println("\nAfter starting:");
        System.out.println("t1 is alive? " + t1.isAlive()); // true (running)
        System.out.println("t2 is alive? " + t2.isAlive()); // true (running)
        try {
            // wait for both threads to finish
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            System.out.println(e);
        }
        System.out.println("\nAfter join:");
        System.out.println("t1 is alive? " + t1.isAlive()); // false
        System.out.println("t2 is alive? " + t2.isAlive()); // false
        System.out.println("\nMain thread finished.");
    }
}
```

Output:

Before starting:

t1 is alive? false

t2 is alive? false

After starting:

t1 is alive? true

t2 is alive? true

Thread-1 - Count: 1

Thread-0 - Count: 1

Thread-0 - Count: 2

Thread-1 - Count: 2

Thread-0 - Count: 3

Thread-1 - Count: 3

After join:

t1 is alive? false

t2 is alive? false

Main thread finished.

Result:

Thus the program illustrating is Alive and join () is successfully executed.

C) Write a Program illustrating Daemon Threads.

Aim: To write a Program illustrating Daemon Threads

Program:

```
class MyDaemonThread extends Thread {
    public void run() {
        if (isDaemon()) {
            System.out.println(getName() + " is a Daemon thread.");
        } else {
            System.out.println(getName() + " is a User thread.");
```

```
        }
        for (int i = 1; i <= 3; i++) {
            System.out.println(getName() + " - Count: " + i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}
public class Main {
    public static void main(String[] args) {
        MyDaemonThread t1 = new MyDaemonThread();
        MyDaemonThread t2 = new MyDaemonThread();
        t1.setDaemon(true);  // set t1 as daemon
        t1.setName("Daemon-Thread");
        t2.setName("User-Thread");
        t1.start();
        t2.start();
    }
}
```

Output:

```
Daemon-Thread is a Daemon thread.
User-Thread is a User thread.
User-Thread - Count: 1
Daemon-Thread - Count: 1
Daemon-Thread - Count: 2
User-Thread - Count: 2
Daemon-Thread - Count: 3
User-Thread - Count: 3
```

Result:

Thus the Program illustrating Daemon Threads is sussessfully executed.

D) Write a JAVA program Producer Consumer Problem

Aim: To write a JAVA program for Producer Consumer Problem.

Program:

```
class Customer {
        int amount=10000;
        synchronized void withdraw(int amount) {
          System.out.println("going to withdraw...");
          if(this.amount<amount) {
                  System.out.println("Less balance; waiting for deposit...");
                  try {
                          wait();
                  }
                  catch(Exception e) {}
          }
          this.amount - = amount;
          System.out.println("withdraw completed...");
        }
        synchronized void deposit(int amount) {
          System.out.println("going to deposit...");
          this.amount + = amount;
          System.out.println("deposit completed... ");
          notify();
        }
}
class Main {
        public static void main(String args[]) {
          final Customer c=new Customer();
          new Thread() {
                  public void run() {
                          c.withdraw(15000);
```

```
                }
        } .start();
        new Thread() {
                public void run() {
                        c.deposit(10000);
                }
        } .start();
    }
}
```

Output:

going to withdraw...

Less balance; waiting for deposit...

going to deposit...

deposit completed...

withdraw completed...

Result:

Thus the program for Producer Consumer Problem is successfully executed.

EXPERIMENT-8

A) Write a JAVA program that import and use the user defined packages

Aim: To write a JAVA program that import and use the user defined packages

Program:

```
package mypackage;
public class MyMath {
    public int add(int a, int b) {
        return a + b;
    }
    public int multiply(int a, int b) {
        return a * b;
    }

// Import user-defined package
```

```
import mypackage.MyMath;
public class TestPackage {
    public static void main(String[] args) {
        MyMath math = new MyMath();
        int sum = math.add(10, 20);
        int product = math.multiply(5, 4);
        System.out.println("Sum: " + sum);
        System.out.println("Product: " + product);
    }
}
```

Output:

Sum: 30

Product: 20

Result:

Thus the JAVA program that import and use the user defined packages is Successfully executed.

B) Without writing any code, build a GUI that display text in label and image in an ImageView(use JavaFX)

Aim: To build a GUI that display text in label and image in an ImageView(use JavaFX)

JavaFX GUI Layout Design

1. Root Container

- Use a VBox or BorderPane as the root layout.
- Purpose: Stack elements vertically (Label above ImageView) or position easily.

2. Label

- Drag and drop a Label control.

- Set its text property, e.g., "Welcome to JavaFX!".
- Optional properties:

  Font size: 20

  Text color: blue

  Alignment: Center

  3. ImageView

- Drag and drop an Image View control below the Label.
- Set its image property: select an image file (e.g., welcome.png).
- Optional properties:

  Fit Width: 300

  Fit Height: 200

  Preserve Ratio: true

4. Layout Alignment

- Set the VBox alignment to CENTER.
- Set spacing between Label and ImageView: 10-20 px.

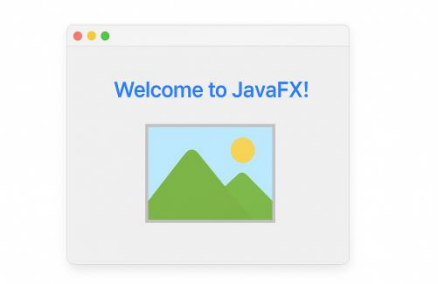
5. Scene

- Scene size: Width 400, Height 400.
- Background: optional color (lightgray or white).

6. Result

- Label will display text at top or center.
- Image View will display the image below the text.
- GUI will look simple and centered, ready to run.

Output:



Result:

Thus to build a GUI that display text in label and image in an Image View(use JavaFX) is successfully executed.

C) Build a Tip Calculator app using several JavaFX components and learn how to respond touser interactions with the GUI

Aim: To Build a Tip Calculator app using several JavaFX components and learn how to respond touser interactions with the GUI

Program:

```
package tipcalculator;

import javafx.application.Application;

import javafx.geometry.Insets;
```

```
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.stage.Stage;

public class tipcalculator extends Application {

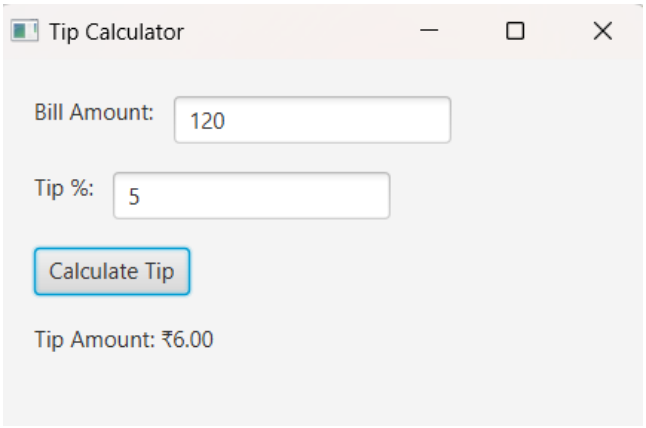
    @Override
    public void start(Stage primaryStage) {
        Label billLabel = new Label("Bill Amount:");
        Label tipLabel = new Label("Tip %:");
        Label resultLabel = new Label("Tip Amount: ");
        TextField billField = new TextField();
        TextField tipField = new TextField();
        HBox billBox = new HBox(10, billLabel, billField);
        HBox tipBox = new HBox(10, tipLabel, tipField);
        HBox resultBox = new HBox(10, resultLabel);
        VBox root = new VBox(15, billBox, tipBox, calculateBtn, resultBox);
       root.setPadding(new Insets(20));
        calculateBtn.setOnAction(e -> {
            try {
                double bill = Double.parseDouble(billField.getText());
                double tipPercent = Double.parseDouble(tipField.getText());

                double tipAmount = bill * tipPercent / 100;
                resultLabel.setText("Tip Amount: ₹" + String.format("%.2f", tipAmount));
            } catch (NumberFormatException ex) {
                Alert alert = new Alert(Alert.AlertType.ERROR);
                alert.setTitle("Input Error");
                alert.setHeaderText("Invalid Input");
                alert.setContentText("Please enter valid numbers for bill and tip percentage.");
                alert.showAndWait();
```

```
        }
    });
    Scene scene = new Scene(root, 350, 200);
    primaryStage.setTitle("Tip Calculator");
    primaryStage.setScene(scene);
    primaryStage.show();
  }

  public static void main(String[] args) {
    launch(args);
  }
}
```

Output:



Result:

Thus to Build a Tip Calculator app using several JavaFX components and learn how to respond touser interactions with the GUI is successfully executed.

EXPERIMENT- 9

A) Write a java program that connects to a database using JDBC

Aim: To write a java program that connects to a database using JDBC

Program:

```
import java.sql.*;


class ConnectDB {
    public static void main(String[] args) {
        Connection con = null;


        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            con = DriverManager.getConnection("jdbc:mysql://localhost:3306/testdb",
"root", "password");


            if (con != null)
                System.out.println(" Connection established successfully!");
            else
                System.out.println(" Connection failed!");


        } catch (Exception e) {
            System.out.println("Error: " + e);
        } finally {
            try {
                if (con != null)
                    con.close();
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }
}
```

Output:

Connection established successfully!

Result:

Thus the program that connects to a database using JDBC is successfully executed.

B) Write a java program to connect to a database using JDBC and insert values into it.

Aim: To write a java program to connect to a database using JDBC and insert values into it.

Program:

```
import java.sql.*;


class InsertRecord {
    public static void main(String[] args) {
        Connection con = null;
        PreparedStatement ps = null;


        try {
            Class.forName("com.mysql.cj.jdbc.Driver");


            con = DriverManager.getConnection("jdbc:mysql://localhost:3306/testdb",
"root", "password");


            String query = "INSERT INTO students (id, name, age) VALUES (?, ?, ?)";
            ps = con.prepareStatement(query);
            ps.setInt(1, 101);
            ps.setString(2, "John");
            ps.setInt(3, 22);
            int rows = ps.executeUpdate();
            if (rows > 0)
```

```
            System.out.println("Record inserted successfully!");
        else
            System.out.println(" Insertion failed!");

    } catch (Exception e) {
        System.out.println("Error: " + e);
    } finally {
        try {
            if (ps != null)
                ps.close();
            if (con != null)
                con.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Output:

Record inserted successfully!

Result:

Thus the java program to connect to a database using JDBC and insert values into it

Is successfully executed.

C) Write a java program to connect to a database using JDBC and delete values from it.

Aim: To write a java program to connect to a database using JDBC and delete values from it.

Program:

```
import java.sql.*;


class DeleteRecord {
    public static void main(String[] args) {
        Connection con = null;
        PreparedStatement ps = null;


        try {
            Class.forName("com.mysql.cj.jdbc.Driver");


            con = DriverManager.getConnection("jdbc:mysql://localhost:3306/testdb",
"root", "password");
            String query = "DELETE FROM students WHERE id = ?";
            ps = con.prepareStatement(query);
            ps.setInt(1, 101);
            int rows = ps.executeUpdate();


            if (rows > 0)
                System.out.println("Record deleted successfully!");
            else
                System.out.println(" No record found with that ID!");


        } catch (Exception e) {
            System.out.println("Error: " + e);
        } finally {
            try {
                if (ps != null)
                    ps.close();
                if (con != null)
                    con.close();
            } catch (SQLException e) {
                e.printStackTrace();
```

```
            }
        }
    }
}
```

Output:

Record deleted successfully!

Result:

Thus the java program to connect to a database using JDBC and delete values from it is successfully executed.